BREAKING RSA

(Before doing anything stay sure that you are connected via vpn)

How many services are running on the box? 1

Nmap (network recognisement) 1

What is the name of the hidden directory on the web server? (without leading '/') 2

FFUF 2

What is the length of the discovered RSA key? (in bits) 4

What are the last 10 digits of n? (where 'n' is the modulus for the public-private key pair) 4

Factorize n into prime numbers p and q | What is the numerical difference between p and q? 5

What is the flag? 6

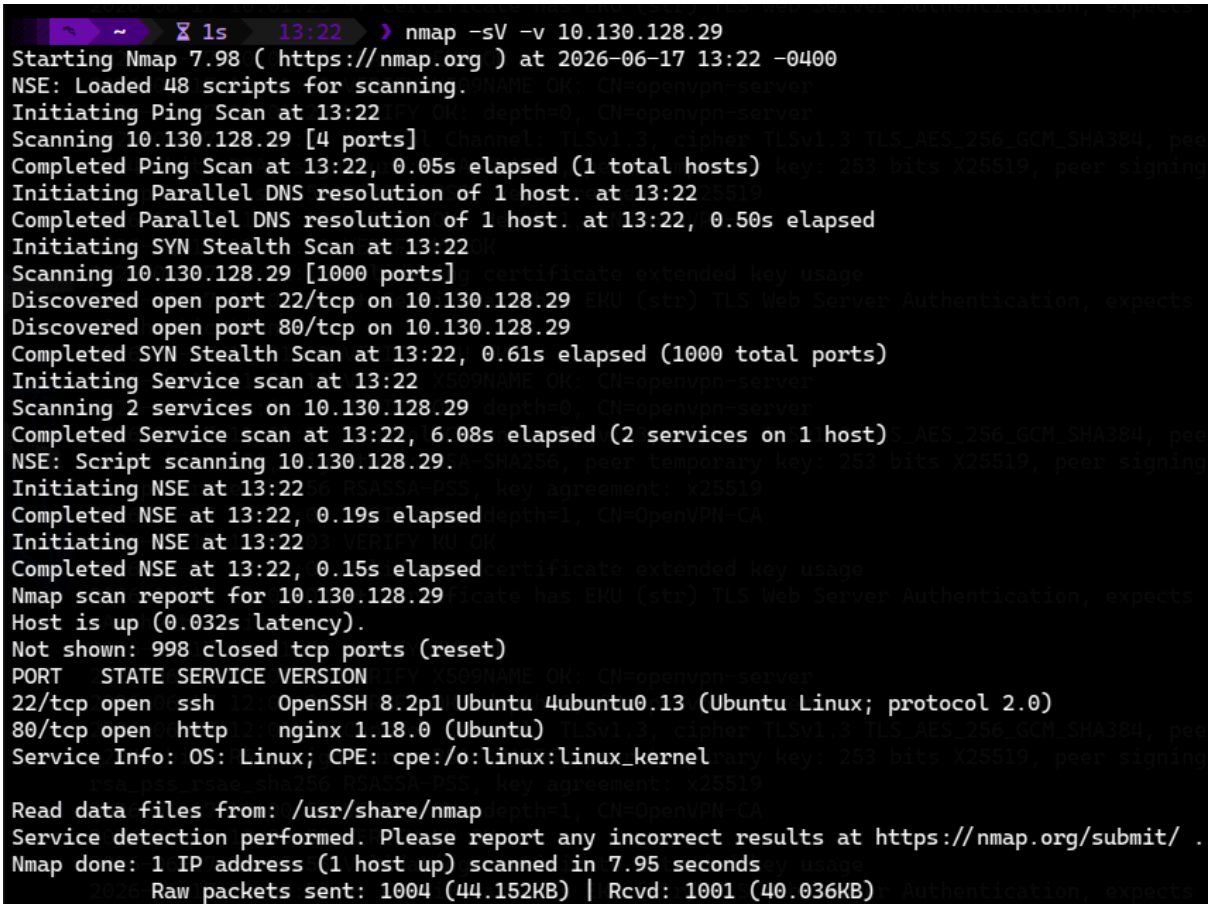
How many services are running on the box?

Nmap (network recognisement)

At first i did an nmap to look what ports and service are up, using -sV and -v that add extra information.

Then we see that the are 2 ports up 22 for ssh and 80 for http/nginx.

Before that we know 2 things we can connect to the machine via ssh when we know the pass or brute forcing it and that we can go to the http domain and try take more information like hidden directories.



What is the name of the hidden directory on the web server? (without leading '/')

- FFUF

  If we dont have any list for search hidden directories we have to install it previously

```
~ ⧖ 0s 13:24 ❯ sudo apt install seclists
[sudo] password for kali:
Installing:
  seclists

Summary:
  Upgrading: 0, Installing: 1, Removing: 0, Not Upgrading: 1541
  Download size: 545 MB
  Space needed: 1,935 MB / 55.6 GB available

Get:1 http://kali.download/kali kali-rolling/main amd64 seclists all 2025.3-0kali1 [545 MB]
Fetched 545 MB in 1min 52s (4,874 kB/s)
Selecting previously unselected package seclists.
(Reading database… 426148 files and directories currently installed.)
Preparing to unpack …/seclists_2025.3-0kali1_all.deb…
Unpacking seclists (2025.3-0kali1)…
Setting up seclists (2025.3-0kali1)…
Processing triggers for kali-menu (2026.1.5)…
Processing triggers for wordlists (2026.1.2)…
```

  Now we can use the ffuf tool to search hidden directories

```
~ ⧖ 0s 13:38 ❯ ffuf -u http://10.130.128.29/FUZZ -w /usr/share/seclists/Discovery/Web-Content
/common.txt

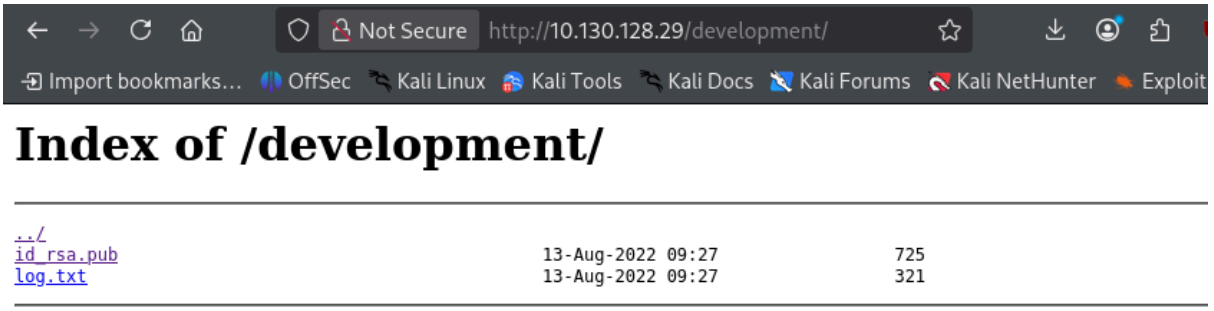
        /'___\  /'___\           /'___\
       /\ \__/ /\ \__/  __  __  /\ \__/
       \ \ ,__\\ \ ,__\/\ \/\ \ \ \ ,__\
        \ \ \_/ \ \ \_/\ \ \_\ \ \ \ \_/
         \ \_\   \ \_\  \ \____/  \ \_\
          \/_/    \/_/   \/___/    \/_/

       v2.1.0-dev
________________________________________________

 :: Method           : GET
 :: URL              : http://10.130.128.29/FUZZ
 :: Wordlist         : FUZZ: /usr/share/seclists/Discovery/Web-Content/common.txt
 :: Follow redirects : false
 :: Calibration      : false
 :: Timeout          : 10
 :: Threads          : 40
 :: Matcher          : Response status: 200-299,301,302,307,401,403,405,500
________________________________________________

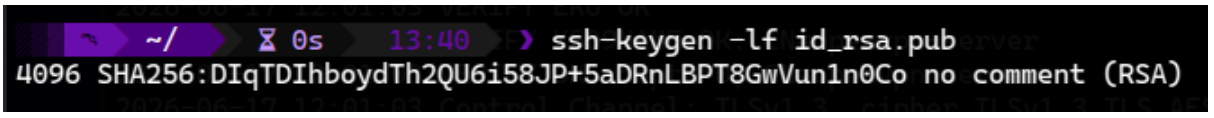
development             [Status: 301, Size: 178, Words: 6, Lines: 8, Duration: 31ms]
index.html              [Status: 200, Size: 384, Words: 113, Lines: 22, Duration: 36ms]
:: Progress: [4750/4750] :: Job [1/1] :: 1162 req/sec :: Duration: [0:00:04] :: Errors: 0 ::
```

  I used to try common.txt and it was successful we found the development directory that contains the public key from the ssh

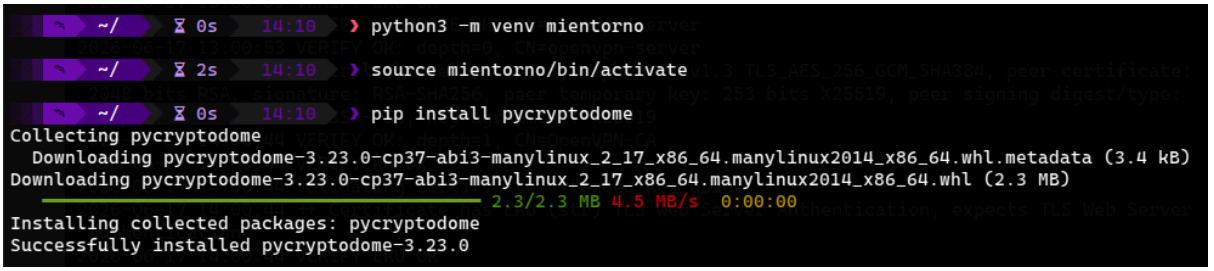


What is the length of the discovered RSA key? (in bits)

Now we can see how many bits is the length of the key with the next command.



What are the last 10 digits of n? (where 'n' is the modulus for the public-private key pair)



In my case i installed pycryptodome in a venv because PEP668 in debian/kali

```python
from Crypto.PublicKey import RSA

with open("id_rsa.pub", "rb") as f:
    key = RSA.import_key(f.read())

print(str(key.n)[-10:])
```

Then i used this script in python to extract the last ten digits of n

```
~/ ⧖ 1s 14:11 ❯ python3 sacar_n.py
1225222383
```

Factorize n into prime numbers ρ and q | What is the numerical difference between ρ and q?

```python
from Crypto.PublicKey import RSA

with open("id_rsa.pub", "rb") as f:
    key = RSA.import_key(f.read())

print(str(key.n))
```

I edited the previous script to it give us ALL n

```
~/ ⧖ 18s 14:16 ❯ python3 n.py
96034377877554948880671622968802256269246318546066431455981951165725529218082720917462405969006062971551318052773416079818503495888365070972703219077208495911625966404792271542752208935372795266682443320758544039581341847167877557299542224800810846298079055847699336291963951612053836251692762231518727497173408143523007915320575075102064295675711703085205300814697656053158344700335513546035992885701019624149760424915137435365349168421481367813639664170694912845352656665112316213880689811602792091825813671342737677561872513645198489630078846560491474187297017386854194067540032500667966203078757098669524390301792312110548393533428978383066426072270467347168847035526889805841436674278172558037718014454197880900528173123260416293601555428927447152303866676099426031582998223064066881125044703000346231774060320457712398561871868783301533255448883608789808414723660989303212117229236863767234940525477258174288343164837605293733299563014179392865499007896747519472415182168911702601044530537574860475711627135349840331840954751505883844761853781118291719845417216107224702109957263870046150743283124894478146551141430877037618276636616074813653269380500231672884287651909139940867222267305884455405843116147430862468349122522238 3
```

> If we use some page like FactorDB we may see that N is already registered in the DB and the page give us P and Q.
> (As a sanity check, the 4096-bit key length is consistent with the 1233-decimal-digit modulus shown by FactorDB, confirming we're working with the correct number before proceeding with factorization)

Result:		
status (?)	digits	number
FF	1233 (show)	9603437787...83<1233> = 3098941397...21<617> · 3098941397...23<617>

Once we have the full values of p and q, we calculate the difference between them:

p - q = -1502

The fact that the difference between p and q is this small (compared to their size, over 600 digits each) confirms that this RSA key is vulnerable to Fermat's factorization attack, where the two primes are deliberately chosen too close to each other, making n trivial to factorize.

Generate the private key using p and q (take e = 65537)

> Now we need to do another .py to generate the private key, knowing the RSA private key concepts of how them are formed. Then we only need to put the values on p, q, e and execute .py




```
from Crypto.PublicKey import RSA

p = 309894139792211864408755379621435882790791806572767857734831630848407874317519250084093827820248373350547
q = 309894139792211864408755379621435882790791806572767857734831630848407874317519250084093827820248373350547
e = 65537

n = p * q
phi = (p - 1) * (q - 1)
d = pow(e, -1, phi)

key = RSA.construct((n, e, d, p, q))
print(key.export_key().decode())
```

To generate the private key, we need to understand the underlying RSA math:

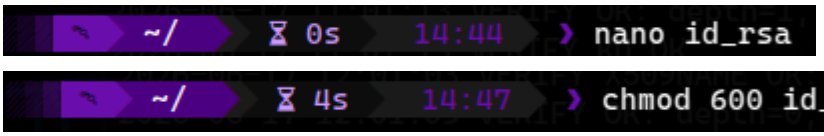
- ϕ(n) (Euler's totient function) is calculated as (p - 1) × (q - 1). It represents how many integers smaller than n are coprime with n.
- The private exponent d is the modular inverse of e with respect to ϕ(n), meaning the value that satisfies (e × d) mod ϕ(n) = 1.
- Once we have n and d, we have the complete private key needed to decrypt any message encrypted with the corresponding public key.

In Python, this is straightforward thanks to the pow() function with three arguments (e, -1, phi), which computes the modular inverse directly without manually implementing the extended Euclidean algorithm.



What is the flag?

Once we have the private key and previous know that the root user is enable we only have to create the private key archive and give it the needed permissions, then we can connect using the private key a search for the flag



Although I didn't keep a record of the contents of log.txt (found earlier alongside id_rsa.pub), I assumed the target user was root, since it's a common setup in TryHackMe machines of this type. This assumption turned out to be correct.

